Projet Billetterie : découpage du travail

Résumé pour l'équipe - 5 personnes - 14 jours - version du 15 octobre 2026

1. Objectif

Livrer en 14 jours une plateforme de billetterie en ligne multi-agences (Cameroun) : recherche de voyages, réservation avec blocage de 5 minutes, paiement simulé, billet PDF avec QR code, vérification à l'embarquement, annulation, rappels par email et commissions. **Tout le monde travaille en même temps dès le premier jour :** le front code contre le contrat **docs/API.md** avec des données simulées, le back teste avec la base de démonstration.

2. Calendrier

Jours	Phase	Contenu	Jalon
J1-J2	Sprint 0	Structure, schéma, contrat d'API (fait)	-
J3-J7	Sprint 1	Le catalogue de voyages : connexion, agences, gares, véhicules, trajets, voyages, recherche	J7 : intégration, tag v0.1, 1re version sur le serveur
J8-J12	Sprint 2	Réserver, payer, billet PDF + QR, vérifier, annuler, rappels, commissions	J12 : intégration, tag v0.2
J13-J14	Sprint 3	Stabilisation : corrections, déploiement, tests de production	J14 : tag v1.0 et démo

3. Qui fait quoi

Personne	Sprint 1 (J3-J7)	Sprint 2 (J8-J12)
Paulo intégration et déploiement	GitHub (main et develop), API v0.2, schéma (table de paramètres), issues, revues de PR, intégration J7	Revues de PR, intégration J12
Balla Back A	Connexion JWT, envoi de fichiers (logos et photos), agences (super admin), gares, véhicules	Paramètres de rappel, remboursements, commissions et taux modifiable, rappels email, billet PDF + QR
Patricia Back B	Villes, agences publiques, recherche de voyages, trajets, voyages, test de concurrence sur les places	Réservation 5 min, paiement, billet et annulation, réservations de l'agence, vérification, expiration
Abdias Front C	Connexion, agences (avec logo), trajets, voyages	Réservations de l'agence, scan QR et vérification, commissions et taux, paramètres de rappel, remboursements
Bouba Front D	Recherche, gares, véhicules (avec photo), carte (option)	Réservation avec décompte de 5 min, paiement, billet (PDF et annulation)

À livrer en premier : la connexion (J4), la recherche et l'envoi de fichiers (J5). En attendant, le front utilise des données simulées (`USE_MOCK=true`).

4. Sprint 3 : stabilisation (J13-J14)

Aucune nouvelle fonctionnalité : le code est gelé au début de J13. Paulo corrige les bugs, fait l'intégration finale et déploie. Toute l'équipe teste la version déployée.

- Chacun teste d'abord son domaine, puis refait un parcours écrit par un autre (test croisé).
- Parcours à tester : voyageur (recherche, réservation, paiement, billet PDF, annulation), admin d'agence (gare, véhicule, trajet, voyage, scan de billet), super admin (agence, commissions), test de charge sur les places (50 clients pour 5 places : exactement 5 billets).
- Chaque bug trouvé devient une issue GitHub étiquetée `bug-prod` (étapes, résultat obtenu, résultat attendu, capture).
- **La v1.0 est prête quand :** les parcours voyageur et agence marchent sur le serveur et sur un vrai téléphone, aucune place n'est vendue deux fois, aucun bug bloquant n'est ouvert, aucun secret n'est dans le dépôt, et les images s'affichent partout.

5. Règles de travail

- **Une branche par tâche** : `feature/<module>-<prénom>`, créée depuis `develop`. Jamais de push direct sur `main` ou `develop`.
- **Une PR par fonctionnalité, petite** (moins de 300 lignes), avec `Closes #numéro`. Relue par un coéquipier du même domaine (Balla et Patricia, Abdias et Bouba), puis fusionnée par Paulo sous 24 h.
- **Un seul propriétaire par fichier.** Les fichiers partagés (pom.xml, pubspec.yaml, application.properties, schema.sql, main.dart, API.md) ne sont modifiés que par Paulo.
- **Le contrat d'API ne change que par une issue** étiquetée `api-change` : Paulo répond sous 24 h et annonce la nouvelle version.
- **Récupérer develop chaque matin** avant de coder. Aucun secret dans Git (mots de passe, clés, jetons).
- **Une tâche est terminée** quand : elle respecte API.md, ses tests passent (mvn test ou flutter test), la case du TODO est cochée, et la PR est relue.

6. Les images doivent s'afficher sur le front

Le logo d'une agence et la photo d'un véhicule sont enregistrés sur le serveur ; la base ne garde que leur chemin. Le back renvoie l'**URL complète** dans les champs `logo` et `image` et le front l'affiche directement. Une tâche d'envoi de fichier n'est terminée que si l'image apparaît à l'écran : logo dans la recherche, la liste des agences, la carte et le billet PDF ; photo dans la liste des véhicules et dans les résultats de recherche. Paulo le vérifie à chaque intégration (J7, J12, J13).

7. Décisions validées

Sujet	Décision
Réservations en attente	Maximum 3 par téléphone et par IP
Tri des résultats	L'heure d'abord, puis la distance (si la position est envoyée)
Agence désactivée	Absente du public et son compte ne peut plus se connecter
Commission	0,7 % par défaut, modifiable par le super admin depuis l'interface ; le changement ne touche que les réservations payées ensuite
Paiement	Simulé uniquement : Campay est abandonné (pas de compte)

8. Suivi sur GitHub (sans Trello)

GitHub Projects remplace Trello : une issue par tâche (personne assignée, étiquettes back, front, option), trois jalons (Sprint 1 le J7, Sprint 2 le J12, Stabilisation le J14) et un tableau Kanban (À faire, En cours, En revue, Terminé). Une PR qui contient `Closes #12` fait passer l'issue en Terminé à la fusion.

Rituels : planification de 30 min (J3 et J8), point quotidien de 10 min (hier, aujourd'hui, blocages), démo après chaque intégration (J7 et J12), rétrospective de 15 min (J7, J12, J14).

9. Si le temps manque, on coupe dans cet ordre

1. La carte et la proximité 2. Les modèles de voyage récurrents 3. La réutilisation du billet